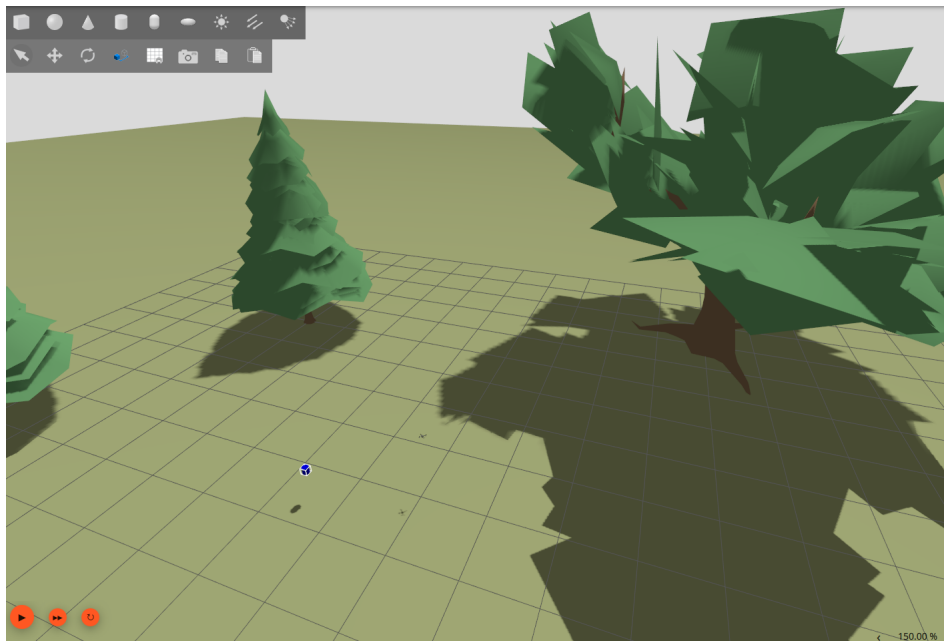




Spring Semester 2025-2026, MT-RO, DISAL-SP208	Start: 16.02.2026
	Finish: 17.06.2026

Using LiDAR to Localize and Track MAVs

Amandine Meunier



Professor:	Alcherio Martinoli
Assistant:	Yacine Derder

Contents

1	Introduction	4
2	Litterature review	5
2.1	LiDAR technologies	5
2.1.1	Mechanical LiDARs	5
2.1.2	Solid-state LiDARs	6
2.1.3	Non-repetitive scanning LiDARs	6
2.2	LiDAR-based drone localization and tracking	7
2.3	Position of this work	7
3	Method	8
3.1	Gazebo Sim	8
3.1.1	World	8
3.1.2	Drone model	9
3.1.3	LiDAR	9
3.2	Gazebo Sim - ROS2 bridges	9
3.3	ROS2	10
3.3.1	Localization for a single drone	11
3.3.2	Localization and tracking for multiple drones	16
3.3.3	Drone controller	18
3.3.4	Supervisor	19
3.3.5	Rviz2 visualization	20
4	Experimental setup	22
4.1	Metrics	22
4.1.1	Single drone	22
4.1.2	Multiple drones	24
4.2	Grid search	24
4.2.1	Parameters	24
4.2.2	Setup	25
4.3	Cluster	25

5	Results	27
5.1	Results for one trajectory	27
5.2	Grid search results for single drone localization	29
5.2.1	Performance for different LiDAR specifications	29
5.2.2	Performance for different drone-LiDAR distances	31
5.2.3	Performance for different drone speeds	35
5.3	Results for multi-drone tracking	37
6	Conclusion	40
A	Code	42
	Bibliography	43

Chapter 1 Introduction

The goal of this project is to localize and track Micro Aerial Vehicles (MAVs) using a fixed LiDAR. LiDARs are increasingly used in robotics, and more specifically drone tracking, for their accuracy and robustness to lighting conditions.

This project consists in designing a localization algorithm taking as input a LiDAR's point cloud, and creating a simulation framework in order to test it. The simulation is done in Gazebo Sim, it includes a Crazyflie drone and a LiDAR, as well as some background elements. A drone controller is also designed for the tests, and some metrics are defined to evaluate the algorithm's performance.

Once the algorithm is functional in the simulated environment, the second step of the project is to implement a grid search to test it with varying parameters. The objective is to provide insight into the suitability of different LiDAR specifications for ground based tracking, depending on the experiment conditions.

Additionally, the localization algorithm and simulation environment was extended to handle multi-drone tracking.

This project was carried out as a semester project with DISAL, at EPFL.

Chapter 2 Litterature review

LiDAR (Light Detection and Ranging) sensors have become more and more popular in robotics due to their ability to provide accurate three-dimensional measurements independently of ambient lighting conditions. Compared to camera-based systems, LiDAR sensors are generally more robust to illumination changes, shadows, and glare, making them particularly useful for outdoor applications such as drone tracking.

2.1 LiDAR technologies

A LiDAR estimates the distance to surrounding objects by emitting laser pulses and measuring the time required for the reflected light to return to the sensor. The resulting measurements are combined to generate a 3D point cloud representing the environment.

Several LiDAR technologies, as seen in **Figure 2.1**, are currently available, each with different trade-offs in terms of field of view, resolution, update rate, reliability, and cost.

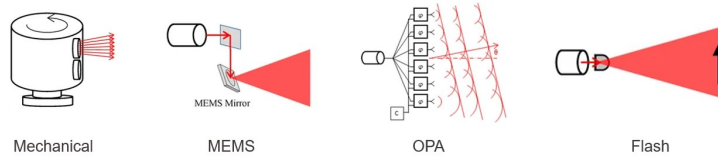


Figure 2.1: Different types of LiDAR technologies.

2.1.1 Mechanical LiDARs

Mechanical LiDARs use rotating components to move laser beams and scan the environment. Classical spinning LiDARs, such as the Ouster OS series or the Velodyne sensors, perform a complete 360° horizontal sweep while simultaneously measuring multiple vertical rays.

The main advantages of mechanical LiDARs are their large field of view and far range. They are well suited for localization applications where drones may appear anywhere around the sensor. However, the presence of moving

mechanical parts increases complexity, size, and long-term wear (prone to mechanical failure).

2.1.2 Solid-state LiDARs

Solid-state LiDARs do not have large rotating components and instead rely on alternative beam steering technologies. Several approaches exist:

- **MEMS LiDARs:** use micro-electromechanical mirrors oscillating to steer the laser beam.
- **Flash LiDARs:** illuminate the entire scene in one single exposure and measure depth using an array of detectors.
- **Optical Phased Array (OPA) LiDARs:** electronically steer the beam by adjusting the phase of the wave.

Solid-state sensors are generally smaller, lighter, and more robust. Their price can be more or less expensive than mechanical LiDARs depending on the complexity and quality. However, they often provide a reduced field of view and range compared to mechanical LiDARs. They can also have a non-uniform scanning patterns.

Figure 2.2 provides a general comparison of the different types of LiDARs, but those properties can vary a lot depending on specific models.

	Mechanical	MEMS	OPA	Flash
Range	Long (~200m)	Medium/long (150-200m)	Short/medium (100-150m)	Short (30-80m)
FOV	Surround (360°)	Medium (max 120°)	Narrow (30-90°)	Wide (120-150°)
Resolution	High	High	Medium	Very high
Update rate	Low/medium (10Hz)	Medium (30Hz)	High (>60Hz)	Medium/high (50Hz)
Cost	(300-15,000\$)	Low (500-2,000\$)	High (2,000-10,000\$), should ~	Low (300-1,000\$)

Highly variable

Figure 2.2: General comparison of LiDAR technologies.

2.1.3 Non-repetitive scanning LiDARs

A more recent category of LiDARs is represented by sensors such as the Livox series[1]. Instead of scanning the environment periodically, these sensors generate non-repetitive scanning patterns that progressively fill the field of view over time, making the point cloud denser. With this pattern, the point cloud resolution improves over the integration time, but some blurring effects also appear.

2.2 LiDAR-based drone localization and tracking

Tracking small drones using a LiDAR is challenging because drones occupy only a small portion of the point cloud and may generate very few returns, particularly at long distances. Consequently, most tracking systems combine several processing stages, such as background subtraction, clustering, and state estimation and tracking[2]. Other approaches also include using a LiDAR as camera sensor, using the signal image to detect the ROI with object recognition models and then studying this ROI on the LiDAR point cloud[3]. Another recent approach, that inspired the estimation memory implemented in this project, is the exploitation of a memory bank to exploit sequential information over multiple previous measurements and view each track as a continuous stream[4].

Multi-drone Tracking :

When several drones are flying simultaneously, the tracking problem becomes more complex because new detections must be correctly associated with existing tracks. Classical solutions rely on data association techniques such as nearest-neighbor matching, Global Nearest Neighbor (GNN), ... Feature tracking can also be done using the Mahalanobis distance[5].

2.3 Position of this work

The present work focuses on evaluating how LiDAR specifications influence drone localization performance, contrary to most research who focus on designing and testing their algorithm on one fixed sensor configuration. It investigates the impact of LiDAR resolution, update rate, and operating distance through an automated grid search methodology in a simulation framework.

Chapter 3 Method

This project is composed of two main parts working together : the Gazebo Sim simulation, a world containing the LiDAR, drone and some background elements ; as well as the ROS2 nodes, which handle all the control and computations. The versions used for this project are Gazebo Harmonic and ROS2 Jazzy.

3.1 Gazebo Sim

3.1.1 World

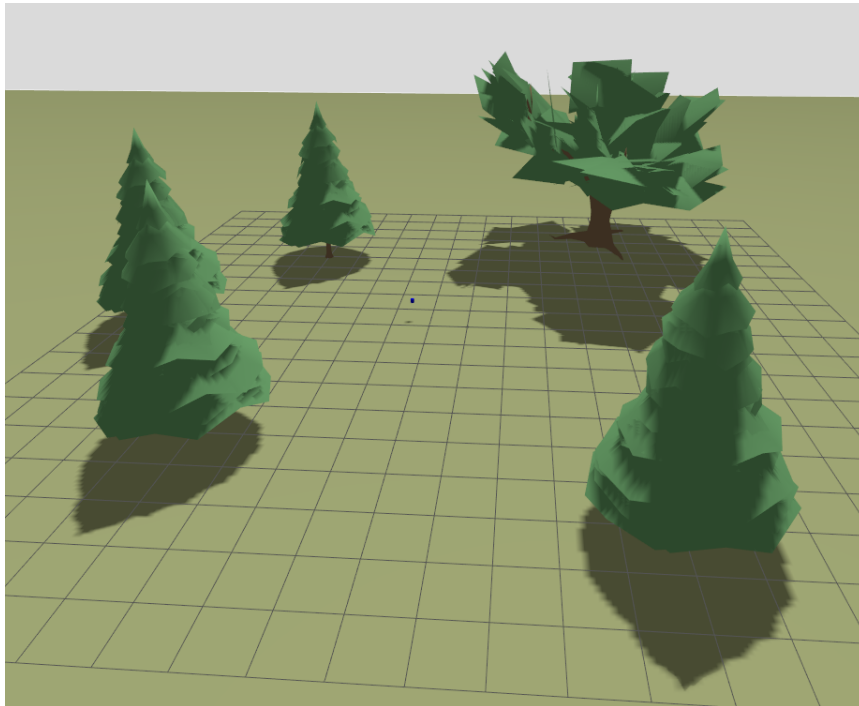


Figure 3.1: Simulation world, The LiDAR is the blue cylinder. 1 grid square is equivalent to one meter.

A Gazebo Sim world can be defined in a *.sdf* file, including models of other entities. The world used for this project contains the drone, the LiDAR, and some background elements to make it more realistic, it is as shown in **Figure 3.1**. The background elements are trees, they are useful to make sure the LiDAR is capable of discerning the drone from the background. Multiple drone models can be included when trying to track multiple drones flying simultaneously.

3.1.2 Drone model

A Crazyflie 2.1 model from bitcraze[6] is used, by cloning a git repository containing a simplified simulation setup for the Crazyflie. This model, shown in **Figure 3.2**, can be included in the simulation world, and expects Twist velocity commands to fly (linear x, y, z and angular x, y, z velocities).



Figure 3.2: Crazyflie 2.1 drone model.

3.1.3 LiDAR

A LiDAR can be simulated in Gazebo, its main parameters are :

- *Update rate [Hz]*
- *Range [m]*
- *Vertical and horizontal field of view [rad]*
- *Vertical and horizontal number of rays*

The LiDAR publishes its point cloud on a Gazebo topic, which can then be retrieved by the ROS2 localization node.

3.2 Gazebo Sim - ROS2 bridges

In order to allow communication between ROS2 and Gazebo Sim, bridges need to be opened for each topic. For this project, a bridge is needed for :

- **Twist commands** : To control the drone (from ROS2 to Gazebo Sim)
- **LiDAR point cloud** : To get the data from the LiDAR (from Gazebo Sim to ROS2)
- **Odometry** : To get the drone's true position, to control the drone and log the information (from Gazebo Sim to ROS2)
- **Clock** : To synchronize Gazebo Sim and ROS2, handle the test durations, and correctly log the gathered information with the correct simulated time (from Gazebo Sim to ROS2)

Multiple Twist and Odometry bridges can be opened when working with multiple drones.

Figure 3.3 shows an overview of how ROS2 and Gazebo Sim are linked.

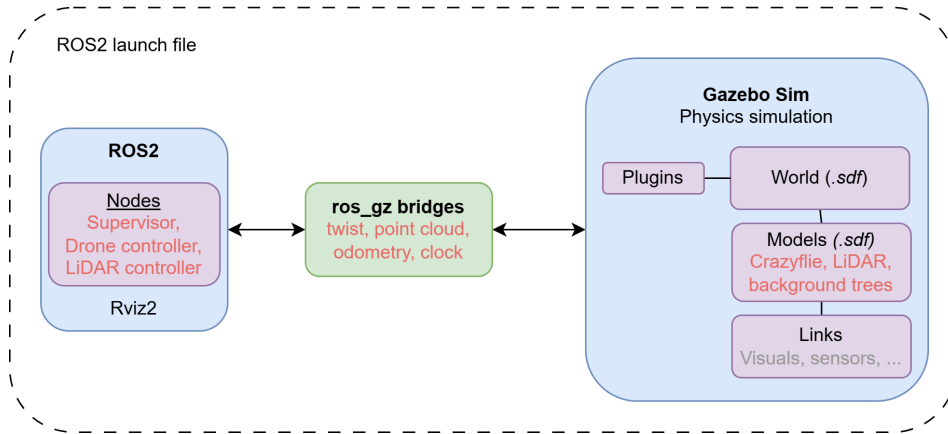


Figure 3.3: Overview of the ROS2 and Gazebo Sim pipeline.

3.3 ROS2

Figure 3.4 explains the general structure and interactions between the ROS2 nodes, and their link to Gazebo entities like the drone and LiDAR.

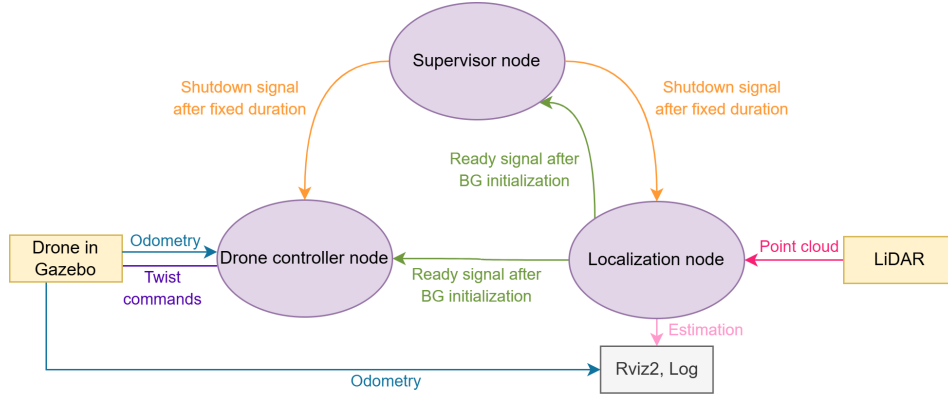


Figure 3.4: Diagram of the different ROS2 nodes and Gazebo Sim entities, and their interactions. The purple ovals are the ROS2 nodes, the yellow rectangles are Gazebo Sim entities, and the arrows correspond to topics.

3.3.1 Localization for a single drone

The localization node estimates in real time the position of a flying drone using the point cloud from the fixed LiDAR. The algorithm combines background subtraction, clustering, motion prediction, and Kalman filtering to compute robust position estimations (as well as velocity and acceleration estimations). The complete pipeline is illustrated in **Figure 3.5**.

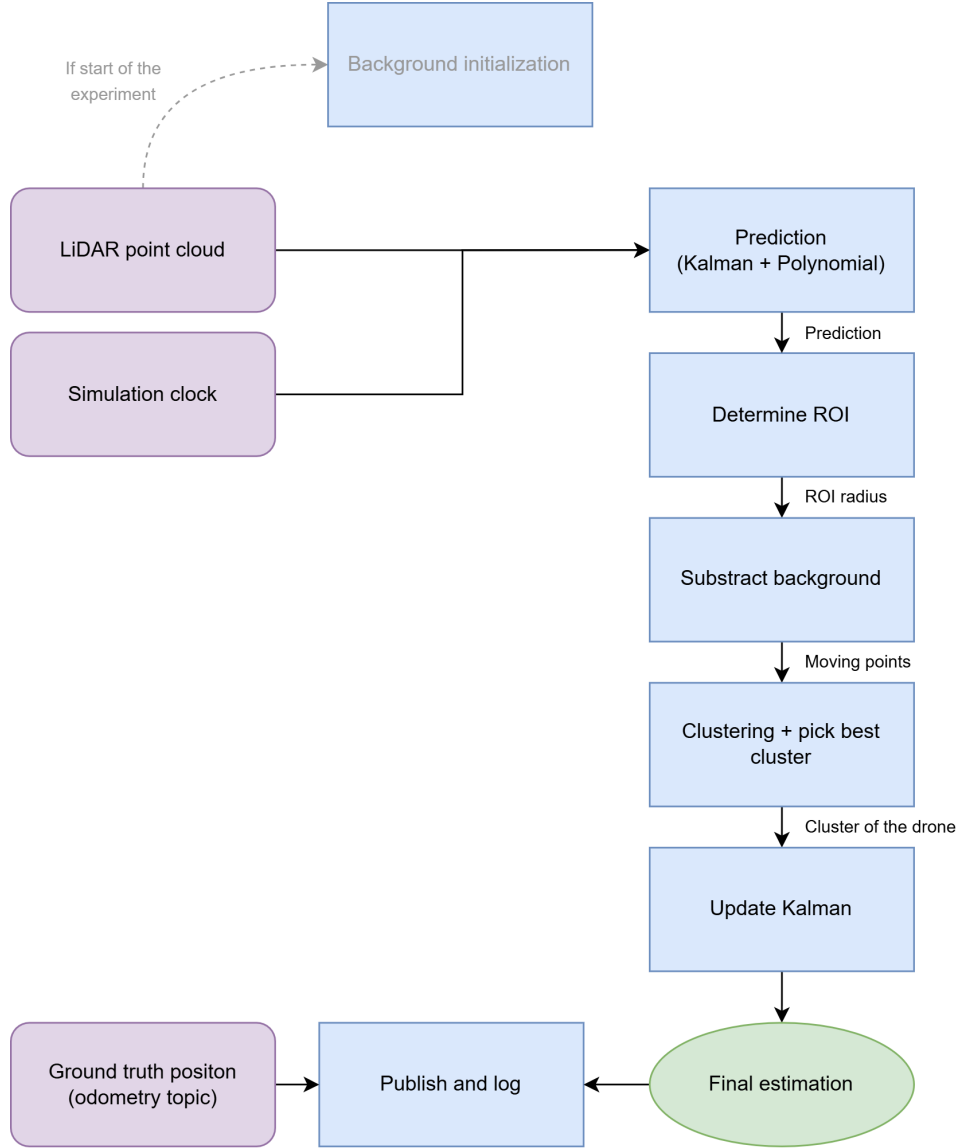


Figure 3.5: Localization pipeline (simplified).

a) Background initialization

At the beginning of the simulation the world is assumed to be static, so multiple LiDAR frames are collected to build a background model. The point clouds from the first 4 measurements are merged in one reference cloud which is then stored in a KD-tree structure to allow efficient searches and analysis later on. Once the background is initialized, the localization node sends a message to the supervisor node and the drone controller node, to inform that the system is ready and can start.

b) Motion prediction and region of interest selection, moving points detection

Once the initialization step is completed, the algorithm can move on to getting new point clouds each time the LiDAR provides an update, to localize the drone. For each point cloud received from the LiDAR, the distance between points from a specific Region Of Interest (ROI) and the same ROI in the background model is computed. The points that are farther from the model than a set threshold (15cm) are classified as moving points, while the others are considered part of the static background. The limited ROI, coupled with the KD-tree structure, is useful to reduce the computational complexity of these computations. At the first simulation step, the ROI is not known yet so the whole point cloud is considered, but later on the ROI is chosen around the predicted position of the drone, so it also improves the robustness of the algorithm to false detections.

Once the drone has been detected at least once, its next position (and the center of the next ROI) is predicted fusing 2 mechanisms : a Kalman filter, and a polynomial trajectory fit.

Kalman filter prediction :

The Kalman filter estimates the drone state, including position, velocity, and acceleration along x , y , and z . The state vector is defined in **Equation 3.1**.

$$\mathbf{x} = [x \ y \ z \ v_x \ v_y \ v_z \ a_x \ a_y \ a_z]^T \quad (3.1)$$

where (x, y, z) is the position, (v_x, v_y, v_z) the velocity, and (a_x, a_y, a_z) the acceleration.

For each point cloud processed, the prediction step propagates the state according to the elapsed time and a constant acceleration motion model, developed in **Equation 3.2** and **Equation 3.3**. Later in the algorithm, the update step incorporates the measured cluster centroid.

$$\mathbf{p}_k = \mathbf{p}_{k-1} + \mathbf{v}_{k-1}\Delta t + \frac{1}{2}\mathbf{a}_{k-1}\Delta t^2 \quad (3.2)$$

$$\mathbf{v}_k = \mathbf{v}_{k-1} + \mathbf{a}_{k-1}\Delta t \quad (3.3)$$

where $\mathbf{p}$ corresponds to the x, y, z position in meters, $\mathbf{v}$ corresponds to the speed along x, y, z in meters per second, and Δt is the elapsed time between the previous step and the current step in seconds. The covariance matrix of the predicted state is also updated.

Polynomial prediction :

The polynomial extrapolation is done using a history of k ($k = 7$) previous

position estimations stored in memory, on which a second-order polynomial is fitted independently for the x , y , and z axis, and evaluated at the next time step.

Final prediction, adaptive ROI :

The final prediction is obtained by combining both results, as written in **Equation 3.4**.

$$\hat{\mathbf{x}} = \alpha * \hat{\mathbf{x}}_{KF} + (1 - \alpha) * \hat{\mathbf{x}}_{poly} \quad (3.4)$$

where $\hat{\mathbf{x}}_{KF}$ is the Kalman prediction, $\hat{\mathbf{x}}_{poly}$ is the polynomial prediction, and $\alpha = 0.65$.

An spherical ROI with an adaptive radius is then centered around the predicted position. The ROI radius is computed from the covariance matrix of the Kalman filter corresponding to the position (not taking into account the velocity and acceleration dimensions), so that the search area gets bigger the more uncertain the algorithm is about the prediction.

c) Clustering

Once the background subtraction has been performed on the ROI, only the points in this ROI that are considered to correspond to moving objects are left. Clustering is performed on them using DBSCAN (Density-Based Spatial Clustering of Applications with Noise). This clustering method identifies clusters based on point density, and the neighborhood radius is chosen according to the known real size of the drone ($DBSCAN_eps = drone_size + margin$). For each clusters found with DBSCAN, their centroid, spatial spread and maximum diameter are computed. Clusters whose diameter exceeds the expected drone size (9.2cm, with a configurable margin) are discarded as they don't correspond to the drone that is being searched for. For each remaining cluster, a score is computed as written in **Equation 3.5**, based on its distance to the prediction, its difference between the previous velocity and the new measured one, and the number of points in the cluster. The score is also penalized if the cluster is too spread.

$$Score = d_p + \alpha * d_v + \beta * s - \delta \log(1 + n) \quad (3.5)$$

with :

d_p : distance between cluster centroid and prediction [m]

d_v : difference in velocity between the previous step and this one [$m.s^{-1}$]

s : cluster spread (mean of the distance between each points of the cluster and the cluster's centroid) [m]

n : number of points in the cluster

This score favors clusters that are spatially close to the prediction, correspond to a smooth motion (no jump in velocity) and contain more LiDAR points. The cluster with the best (smallest) score is kept, its considered to correspond to the drone.

d) Kalman filter update

Filter measurements

To reject illogical measurements before doing the Kalman filter measurement update, the Mahalanobis distance between the predicted state and the new measurement is computed in **Equation 3.6**.

$$D^2 = (\mathbf{z} - \mathbf{H}\mathbf{x})^T \mathbf{S}^{-1} (\mathbf{z} - \mathbf{H}\mathbf{x}) \quad (3.6)$$

with :

$\mathbf{z}$: measurement vector

$\mathbf{H}$: measurement matrix (only keeps position)

$\mathbf{x}$: state vector

$(\mathbf{z} - \mathbf{H}\mathbf{x})$: innovation vector

$\mathbf{S}$: innovation covariance matrix

Measurements whose Mahalanobis distance exceeds a set threshold are rejected as outliers. This makes the algorithm more robust to false positive detections caused by noise in the point cloud.

Measurement update :

The innovation vector, which measures the difference between the observed and predicted positions, is computed in **Equation 3.7**.

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1} \quad (3.7)$$

where $\mathbf{z}$ is the measurement, $\hat{\mathbf{x}}_{k|k-1}$ is the estimated step, and $\mathbf{H}$ is the measurement matrix to only select position (and not velocity and acceleration).

The Kalman gain is obtained as written in **Equation 3.8**.

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^T \mathbf{S}_k^{-1} \quad (3.8)$$

And finally for the measurement update, the predicted state is corrected according to Equation 3.9.

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k \quad (3.9)$$

This correction step combines the motion prediction with the LiDAR observation, and the Kalman gain automatically balances the confidence placed in the prediction versus in the measurement. The covariance matrix of the estimated state is also updated.

e) Drone loss

If no valid cluster is kept in a simulation step, the position is estimated using only the prediction, the measurement update is skipped. When this happens the uncertainty of the Kalman filter increases, which in consequence expands the radius of the ROI for the next step, to increase the search space. This makes the algorithm more likely to find the drone again, but if the drone is lost for too many frames (set threshold), the ROI is set to encompass the whole point cloud.

f) Publishing and logging

Once the position of the drone has been estimated, it is logged at each step in a csv file, along with the time, velocity, acceleration, and lost frame counter, in order to compute performance metrics later on. The estimation is also published on a topic, to visualize in Rviz, along with the ROI.

3.3.2 Localization and tracking for multiple drones

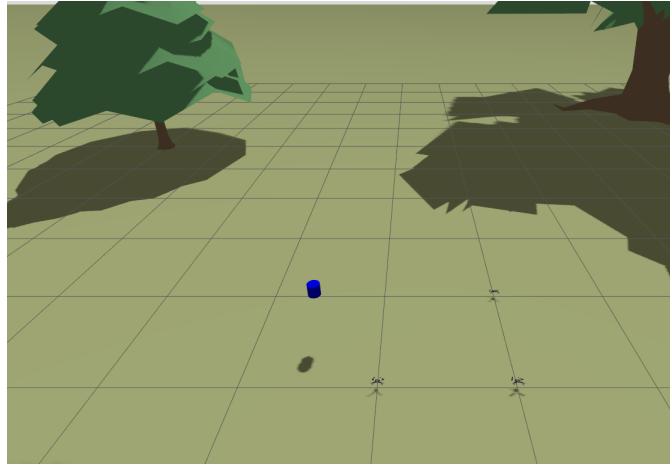


Figure 3.6: 3 drones simulated in Gazebo.

The single-drone localization algorithm presented in **Section 3.3.1** can be extended to localize and track multiple drones at the same time. **Figure 3.6** shows the Gazebo Sim world with 3 drones.

The background initialization, subtraction, and clustering steps remain similar to how it's done for a single drone. However instead of keeping the most fitting cluster, all of the valid clusters are considered as potential drone detections. A track is created for each detected drone, containing its own Kalman filter, trajectory history, state estimations, ROI radius and detection statistics (first seen, number of hits, ...). Then each drone can be tracked independently using the method described in **Section 3.3.1**.

a) Track prediction

At every step, all active tracks are first propagated forward in time. For each track, a position prediction is obtained using the fusion of the Kalman filter prediction and the polynomial trajectory extrapolation described before. This results in a set of predicted positions (**Equation 3.10**).s

$$\mathcal{P} = \hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2, \dots, \hat{\mathbf{x}}_N \quad (3.10)$$

where N is the current number of active tracks.

Each predicted position is associated with an adaptive ROI whose radius is computed from the position covariance of the corresponding Kalman filter. Therefore, each drone maintains its own uncertainty estimate and search area.

b) Detection-Track association

After clustering the new moving points, the algorithm must determine which detected cluster corresponds to which known track. For each detected cluster with all predicted track, a score is computed in **Equation 3.11** to measure how well they fit. But before computing the score, two conditions are applied. First, clusters outside the track's ROI are rejected. Second, clusters whose Mahalanobis distance exceeds the set threshold are discarded. Then, the score is based on the Euclidean distance to the predicted position, the consistency of the measured velocity, the Mahalanobis distance, the cluster spread, and the number of LiDAR points contained in the cluster, similarly to the score to pick the best cluster in **Section 3.3.1**.

$$Score = d_p + \alpha * d_v + \beta * s + \gamma * d_M - \delta * \log(1 + n) \quad (3.11)$$

with :

d_p : distance between cluster centroid prediction $[m]$

d_v : difference in velocity between the previous step and this one $[m.s^{-1}]$

d_M : Mahalanobis distance

s : cluster spread (mean of the distance between each points of the cluster and the cluster's centroid) $[m]$

n : number of points in the cluster

All valid track-cluster pairs are sorted according to their score, and the best association is selected iteratively until no valid pair remains. Each cluster can be assigned to at most one track, and each track can receive at most one cluster (for each timestep, not for all the duration of the simulation).

c) Track update

When a cluster is successfully associated with a track, the track is updated using the Kalman filter measurement update. The corresponding trajectory history, velocity estimate, acceleration estimate, and detection statistics are also updated. The measurement noise covariance is adapted dynamically according to the quality of the detected cluster. Clusters containing many points and having low spatial spread are considered more reliable and therefore result in lower measurement uncertainty.

d) Track creation

Clusters that have not been associated to an existing track are assumed to correspond to newly detected drones. For each of these clusters, a new track is initialized. The centroid of the cluster is used to initialize the state of its Kalman filter. Each newly created track then follows the same prediction and update process as the other existing tracks.

e) Track loss and deletion

If no cluster can be associated with a track for a simulation step, the track relies only on prediction. The Kalman filter keeps propagating the state estimate and the uncertainty increases. A lost counter is incremented for each consecutive missed detection. If a valid measurement is obtained again, the counter is reset and normal tracking resumes. But tracks whose lost counter goes over a predefined threshold are considered permanently lost and are removed from the tracking system.

f) Publishing and logging

Every active track (position estimation and ROI) is published on a dedicated ROS2 topic, so they can all be visualized in Rviz. They are also logged in a csv file, with a unique ID for each track, along with other information such as detection count, velocity, etc, which will be useful to study metrics later on.

3.3.3 Drone controller

The drone controller node generates and tracks a 3D trajectory, publishing Twist commands to control the drone to follow it. It waits for the localization node to be ready (static background initialized) before starting the drone. First a 3D trajectory is generated following **Equation 3.12**.

$$\begin{aligned}x &= L * \cos(\theta) \\y &= W * \cos(\theta) \\z &= h + \Delta z \sin(2\theta)\end{aligned}\tag{3.12}$$

with :

L : maximum trajectory length [m]

W : maximum trajectory width [m]

h : base altitude [m]

Δz : vertical oscillation amplitude [m]

The resulting trajectory resembles an ellipse with a sinusoidal altitude variation, and discrete waypoints are sampled at fixed distance intervals on the trajectory.

In the beginning the drone receives constant vertical speed commands to take off, until it reaches the altitude of $h - \Delta z$. Then the drone can start following the ellipse, using a proportional controller. The node is subscribed to the odometry topic from the simulated drone in Gazebo Sim, which provides the ground truth position of the MAV. Each new velocity commands is computed using the P-controller, as described in **Equation 3.13**, and clipped to stay in defined safety bounds.

$$v = K_p * (\mathbf{x}_{\text{waypoint}} - \mathbf{x}_{\text{drone}}) \quad (3.13)$$

with :

K_p : proportional gain [s^{-1}]

$\mathbf{x}_{\text{waypoint}}$: position of the targeted point [m]

$\mathbf{x}_{\text{drone}}$: position of the drone [m]

At each step the controller checks if the current targeted waypoint is reached to know when to change to the next. A look-ahead offset can also be set to target farther waypoints, to make the drone movements smoother as it reduces the slowdown proportional to the error diminishing when reaching a targeted waypoint. The drone stops following the trajectory and the node shuts down once the end signal is received from the supervisor node.

Multiple drones can be controlled in the same way simultaneously by publishing on multiple Twist topics and subscribing to multiple Odometry topics.

Note : An alternative controller node can be used to control the drone with the keyboard.

3.3.4 Supervisor

The supervisor node is used to control the time of each test run. Once it receives the information that the localization is ready (background initialized), meaning both the localization algorithm and the drone controller are starting, it starts a timer and after a set duration it sends a shutdown signal to the localization and drone controller nodes.

3.3.5 Rviz2 visualization

The LiDAR point cloud, drone position estimation, and adaptative ROI can be visualized in real time with Rviz2 to observe how the algorithm is doing. **Figure 3.7** gives a glimpse of how the LiDAR perceives the simulated world, and **Figure 3.8** shows a closer view of the drone position estimation and the adaptive ROI around it. **Figure 3.9** also shows the Rviz2 visualization for 3 drones flying simultaneously, using the algorithm to localize and track them all.

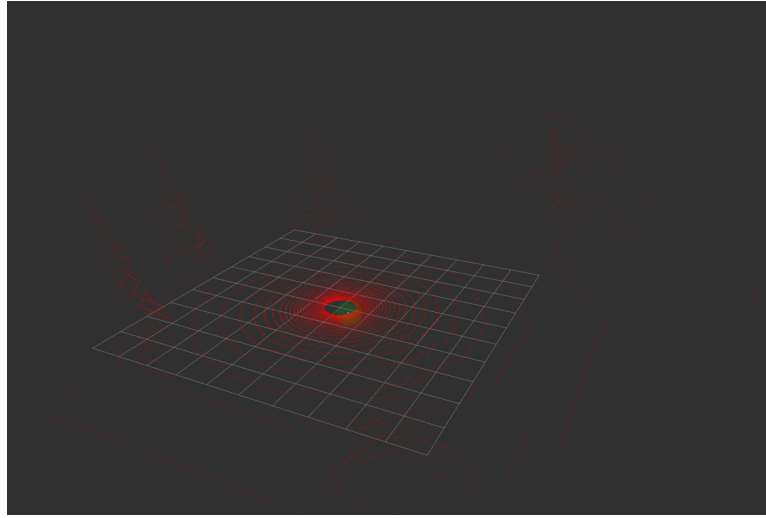


Figure 3.7: Simulated world as seen by the LiDAR (1024×64 rays)

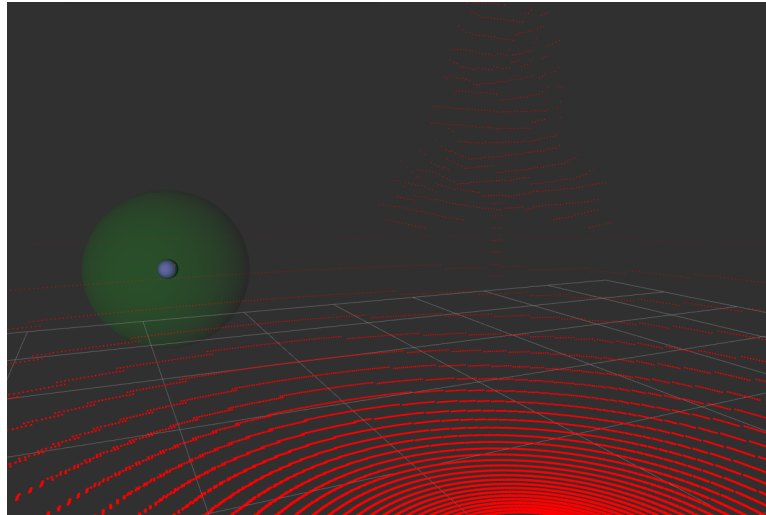


Figure 3.8: Drone position estimation : the purple point represents the estimated position and the green sphere represents the adaptive ROI.

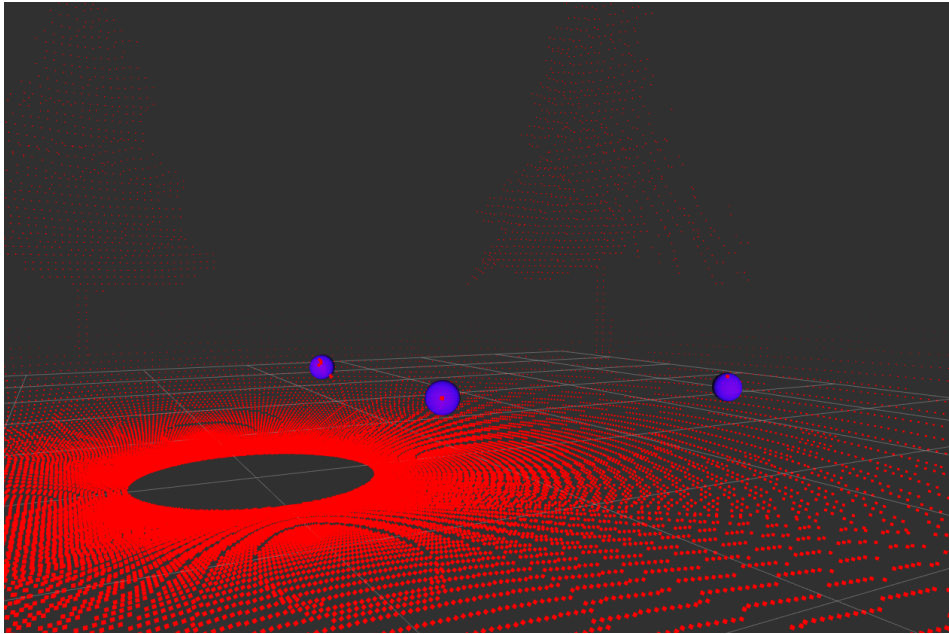


Figure 3.9: Drone position estimation for 3 drones.

Chapter 4 Experimental setup

The goal of this experimental setup is to assess the localization algorithm performance for different LiDAR parameters and drone trajectories. The objective is to see which LiDAR specifications would be adapted depending on the situation.

4.1 Metrics

4.1.1 Single drone

Position error :

The position error is computed at each timestep using the Euclidean distance between the estimated drone position and the ground truth position, as written in **Equation 4.1**.

$$e_{pos} = ||\mathbf{x}_{gt} - \mathbf{x}_{estimated}|| \quad (4.1)$$

If this position error is bigger than $1m$, the measure is not valid (the drone was lost by the algorithm) and it is not taken into account when computing the RMSE and mean position error. It will however be taken into account when computing a specific Loss metric.

RMSE :

The root mean square error is computed using all valid position estimations, according to **Equation 4.2**.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N e_{pos}(i)^2} \quad (4.2)$$

RMSE penalizes large tracking error more strongly than the mean error and is used as the primary performance metric.

Mean position error :

The mean position error is computed in **Equation 4.3**.

$$e_{mean} = \frac{1}{N} \sum_{i=1}^N e_{pos}(i) \quad (4.3)$$

Max position error :

The maximum valid position error is defined in **Equation 4.4**.

$$e_{max} = \max(e_{pos}) \quad (4.4)$$

Mean position error per axis :

Equation 4.3 can also be applied for each individual axis (x, y, z), to identify directional biases.

Loss metric :

The number of frames and time duration during which the position error is above $1m$ is computed. This metric evaluates the robustness and reliability of the localization algorithm.

Velocity and acceleration RMSE :

Ground truth velocity and acceleration can be computed using finite-difference approximation, as described in **Equation 4.5** and **Equation 4.6**.

$$v_{gt}(i) = \frac{\mathbf{x}_{gt}(i+1) - \mathbf{x}_{gt}(i-1)}{t(i+1) - t(i-1)} \quad (4.5)$$

$$a_{gt} = \frac{\frac{\mathbf{x}_{gt}(i+1) - \mathbf{x}_{gt}(i)}{\Delta t_2} - \frac{\mathbf{x}_{gt}(i) - \mathbf{x}_{gt}(i-1)}{\Delta t_1}}{\frac{\Delta t_1 + \Delta t_2}{2}} \quad (4.6)$$

With :

$\mathbf{x}_{gt}(i)$: ground truth position vector (x, y, z) at timestep i [m]

$t(i)$: time at timestep i [s]

Δt : duration of the timestep [s]

Then similarly to **Equation 4.2**, their RMSE can be computed between the estimated velocity and acceleration and the computed one from the ground truth positions.

Mean distance to LiDAR :

The mean distance between the drone and the LiDAR for the whole duration of the test can be computed in **Equation 4.7**, to know for which approximate range the localization algorithm performs well.

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_{gt}(i) - \mathbf{x}_{LiDAR}\| \quad (4.7)$$

Tracking delay :

The delay between the estimated position and ground truth position can be computed. To compute this latency the cross-correlation between the 2 signals can be computed, the maximum value of cross correlation corresponding to the temporal offset.

4.1.2 Multiple drones

When working with multiple drone tracks, a step is added at the beginning to match the multiple estimated tracks to the various ground truth tracks. This is done by computing the mean Euclidean distance over time between each estimated track and all the ground truth drone trajectories, each estimated track is then assigned to the closest ground truth track. There can be multiple estimated tracks for one ground truth drone trajectory, when the track was lost for too long and found again later. Once all the estimated tracks are assigned, we can compute for each of them the same metrics as for single drone localization (position RMSE, ...).

4.2 Grid search

In order to compare the localization algorithm performance for different LiDAR and drone motion parameters, a grid search is implemented. This will allow to select the right LiDAR parameters depending on the situation encountered (distance between the drone and LiDAR, speed of the drone, ...).

4.2.1 Parameters

The grid search can explore 3 parameter groups : the localization algorithm parameters, the drone motion parameters, and the LiDAR parameters.

Localization algorithm parameters

- Background distance threshold : *bg_threshold* [m]
- DBSCAN minimum samples : *dbscan_min_samples*
- Memory length (for polynomial prediction) : *memory_len*
- LiDAR noise standard deviation (virtually added after the point cloud obtention) : *lidar_noise_std*
- Drone size margin : *size_margin* [m]
- Region of interest scaling : *roi_factor*, *roi_max_radius*
- Mahalanobis gating threshold : *mahalanobis_gate*

Trajectory parameters

- Base altitude : *base_altitude* [m]
- Speed limit : *speed_limit* [$m.s^{-1}$]
- Trajectory shape : *max_length*, *max_width*, *max_delta_z* [m]
- Waypoint spacing : *waypoint_spacing* [m]
- Controller proportional gain : k_p [s^{-1}]

LiDAR parameters

- Horizontal resolution : *horizontal_samples*
- Vertical resolution : *vertical_samples*
- Maximum range : *max_range* [m]
- Update rate : *update_rate* [Hz]

4.2.2 Setup

All possible combinations of the parameters presented in **Section 4.2.1** are computed, and each combination is tested on a full test run of a fixed duration. To do this, a ROS2 YAML configuration is created with all the parameters of each combination. The LiDAR sdf file is also automatically rewritten to have the right parameters, modifying the range, rate, and number of rays. For each set of parameters, the ROS2 nodes are started using a launch file, and the simulation is stopped by the supervisor after the set fixed duration. Once the individual run is done, the log file containing all the estimations for this run is loaded, and the metrics mentioned in section 4.1 are computed. After each run, the parameters and associated metrics are stored in a results table, which is re-sorted by position RMSE to order the combinations.

4.3 Cluster

The individual runs and the grid search can both be run on DISAL's cluster. SLURM files handle the usage of the docker, the Micromamba Python environment, sourcing ROS2 and building the code, and running the code. Gazebo Sim and the ROS2 nodes are automatically launched and stopped for each test. This setup simplifies the grid search execution, making it easier to run big batches of experiments.

Note : The experiment can suffer from a computation bottle neck in the position estimation. If the computing power for the localization algorithm is not sufficient to keep up with the update rate of the simulation, the estimations will miss some steps and quality of the results will be greatly impacted. The algorithm needs to be ran on a machine with enough computing power, knowing that the complexity of the localization algorithm grows linearly with the number of LiDAR rays ($\mathcal{O}(n)$ complexity). Therefore the choice of number of LiDAR rays is also dependent on the computing power available. Another solution for testing is to slowdown the simulation speed, but this will not be possible when implementing the algorithm in real life.

Chapter 5 Results

To make the simulation results more realistic, a gaussian noise was added on the LiDAR's point cloud, centered around 0 and of standard deviation equal to $0.02m$. The distribution of the noise is shown in **Figure 5.1**, it is applied on all three coordinates of each points.

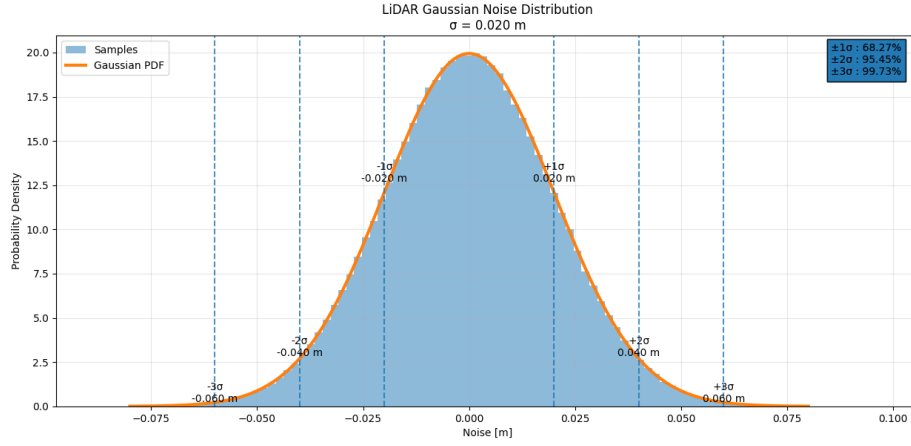


Figure 5.1: Noise distribution added to the LiDAR point cloud (on x , y , and z axis independently).

5.1 Results for one trajectory

Figures 5.2, 5.3, 5.4, and 5.5 show an example of the plots obtained, using a LiDAR with (256×2048) rays updating at $5Hz$, and a drone going at a speed of $0.28m.s^{-1}$, with a mean distance between the drone and the LiDAR of $1.86m$ over the trajectory.

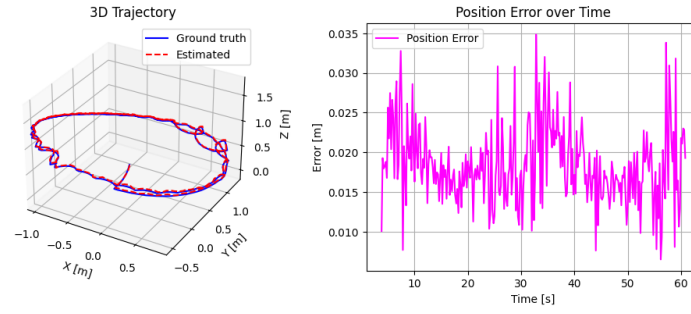


Figure 5.2: Drone trajectory 3D plot, and estimation error.

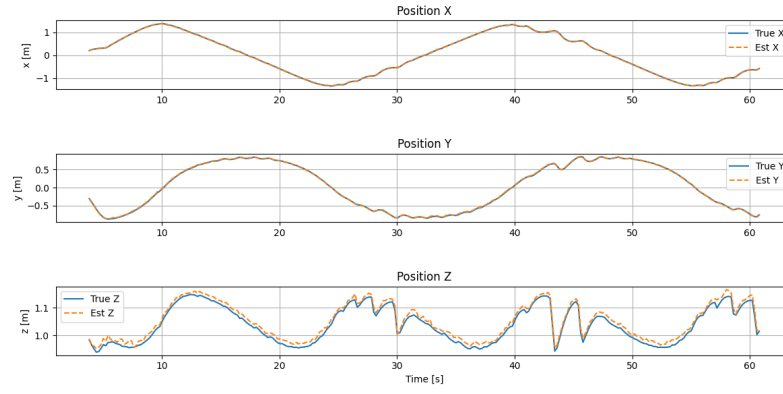


Figure 5.3: Estimated and true position over time, along each axis.

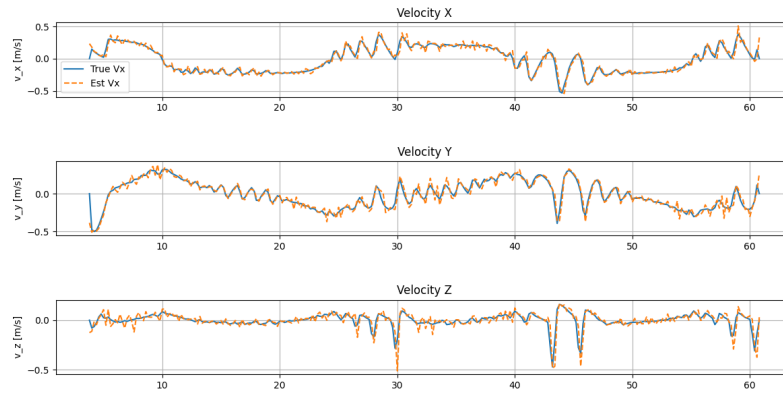


Figure 5.4: Estimated and true velocity over time, along each axis.

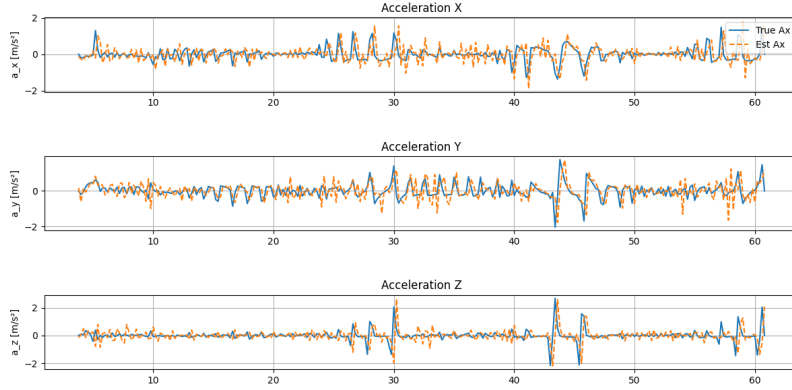


Figure 5.5: Estimated and true acceleration over time, along each axis.

5.2 Grid search results for single drone localization

The results presented in this section were obtained with a simulated LiDAR with a complete horizontal FOV and a vertical FOV of $[-0.79, 0.79]$ ($\pm 45^\circ$), not limited by range ($max_range = 75m$). The LiDAR is placed at an altitude of $0.5m$. The rate of the LiDAR, and its number of vertical and horizontal rays are varied to evaluate the impacts on performance.

The drone follows the oval trajectory as described in **Section 3.3.3**, of length $1.5m$ and width $1.0m$, and base altitude of $1m$ with altitude variations of $0.1m$. Its speed and distance to the LiDAR are also varied.

The 2 main metrics studied are the position RMSE, and the lost ratio (as described in **Section 4.1**). The position RMSE is the root mean square error between the true position of the drone and the estimation computed by the localization algorithm, and the lost ratio is the ratio of simulation steps during which the estimation was wrong by more than $1m$.

5.2.1 Performance for different LiDAR specifications

A first series of tests was performed with the drone flying at a mean distance of $1.94m$ to the LiDAR (ranging from $0.70m$ to $2.67m$), at speeds of $0.18m.s^{-1}$ and then $0.28m.s^{-1}$ (the results for both speeds were averaged in this section, and the comparison between speeds is done in **Section 5.2.3**).

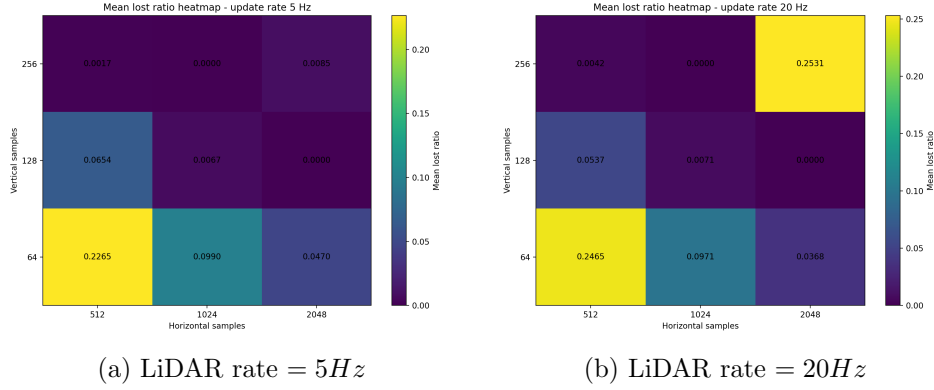


Figure 5.6: Heatmaps of the lost ratio for different pairs of LiDAR rays.

Figure 5.6 shows that the lost ratio improves with the number of rays. For a drone of $9.2cm$ flying at approximately $2m$ of the LiDAR, a pair of (64×512) rays loses tracks of the drone about 23% of the time but this lost ratio becomes negligible when doubling the number of rays on either of the axis (vertical or horizontal). An outlying lost ratio peak can be observed for (256×2048) rays at $20Hz$, which is explained by the localization algorithm not being able to keep up with the simulation speed when working with such large point clouds, and the trajectory controller getting lost too (computing doesn't keep up).

Figure 5.7 also shows the position error (not taking into account the lost frames) improving with the number of rays. A precision of $\sim 23mm$ can be reached with the localization algorithm with at least 262 144 points $((256 \times 1024)$ rays or (128×2048) rays), going up to $36mm$ when lowering the number of rays. The RMSE also gets slightly better with an increasing LiDAR rate, but this also depends on the synchronization between LiDAR rate and estimation computation rate.

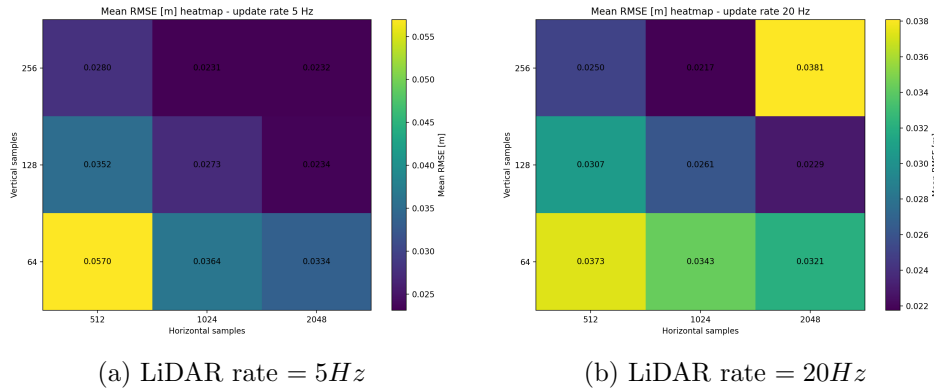
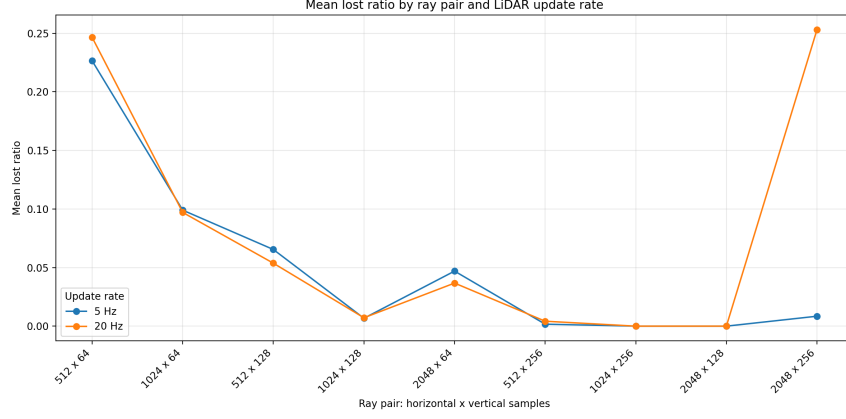
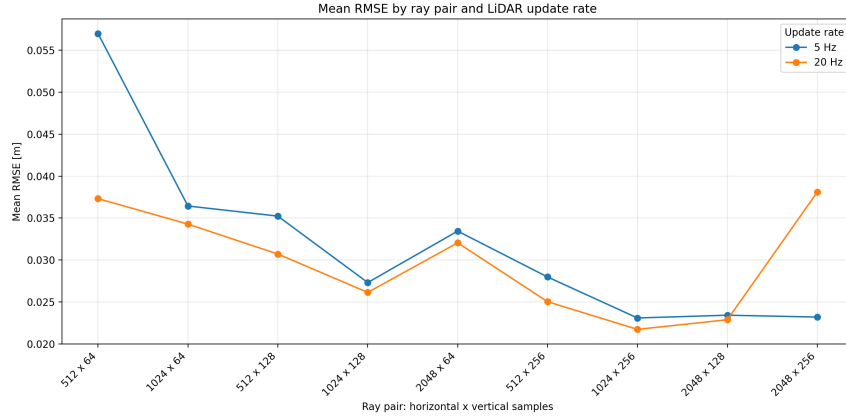


Figure 5.7: Heatmaps of the RMSE for different pairs of LiDAR rays.

The same metrics are presented in the shape of a graph in **Figure 5.8**. The heatmaps allow to visualize best the pairs of number of rays, while the graph is more direct to study the evolution of the values of the metrics.



(a) Lost ratio



(b) RMSE

Figure 5.8: Graphs of the lost ratio and RMSE for different pairs of LiDAR rays.

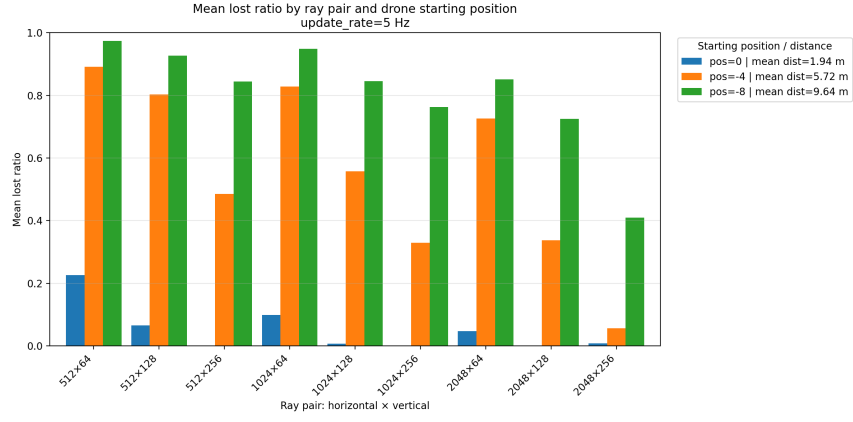
5.2.2 Performance for different drone-LiDAR distances

The localization algorithm was also tested with a second series of tests, with different spacing between the drone and the LiDAR (same trajectory shape but shifted). The main distances describing these variants are described in **Table 5.1**. In all starting position configurations, the drone was flying at speeds of $0.18m.s^{-1}$, and then $0.28m.s^{-1}$ (the results for both speeds were averaged).

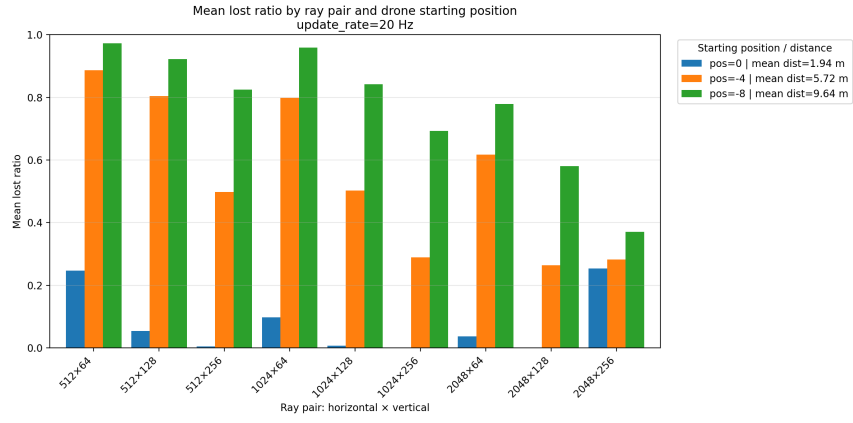
Drone start pos (x, y) [m]	Mean dist [m]	Min dist [m]	Max dist [m]
(0, 0)	1.94	0.70	2.67
(0, -4)	5.72	4.61	6.45
(0, -8)	9.64	8.55	10.43

Table 5.1: Approximate mean, min and max distance between the drone and the LiDAR ($x = 0.5, y = 1.5$) for the whole trajectory depending on the drone starting position.

In **Figure 5.9** and **5.10**, it can be seen that the distance between the drone and the LiDAR significantly impacts the lost ratio. When the drone starts from $(0, -8)$ and has a mean distance to the LiDAR (when considering its whole oval trajectory) of $9.64m$, more than (256×2048) rays would be needed to successfully track it, as the drone is lost around 40% of the time with this LiDAR resolution. However, a drone flying at around $5.72m$ of the LiDAR, with peaks of distance at $6.45m$ can be localized with a lost ratio of 0.051 with (256×2048) rays and an update rate of the LiDAR of $5Hz$. It is the only tested configuration sufficient to work at this distance, when dividing the number of rays by 2 the drone is lost too often to consider the experiment successful. At $20Hz$ with this point cloud size, the algorithm was not able to keep up computationally.

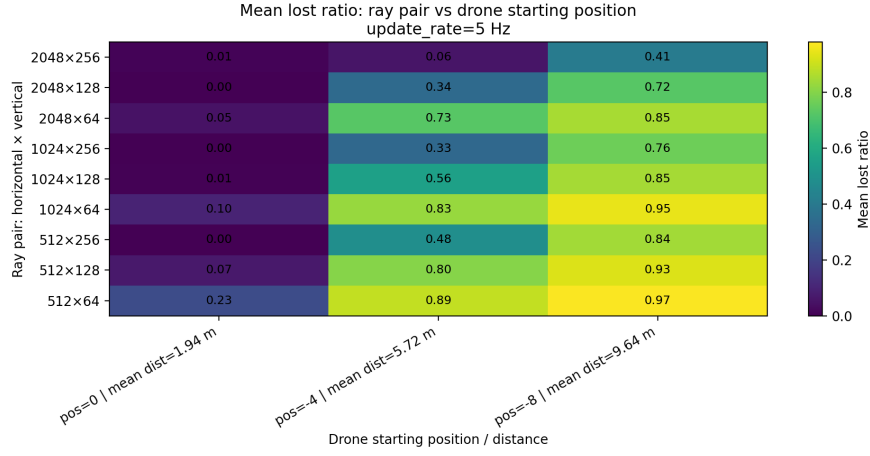


(a) LiDAR rate = $5Hz$

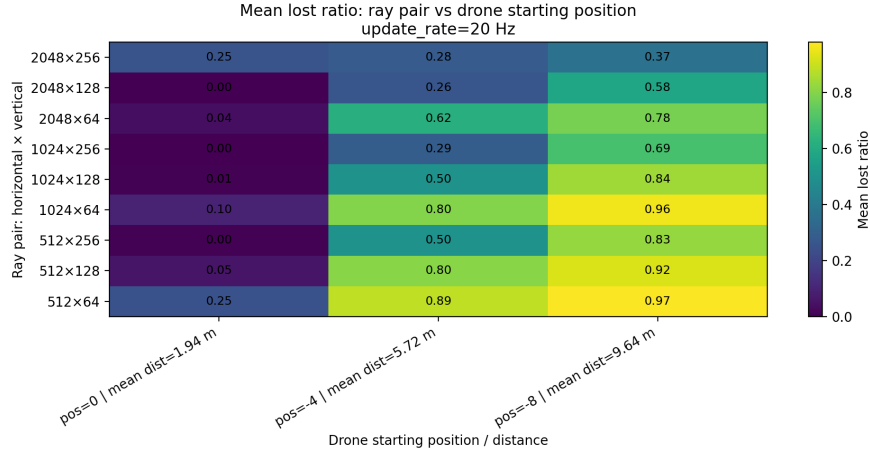


(b) LiDAR rate = $20Hz$

Figure 5.9: Bar graphs of lost ratio for different pairs of LiDAR rays.



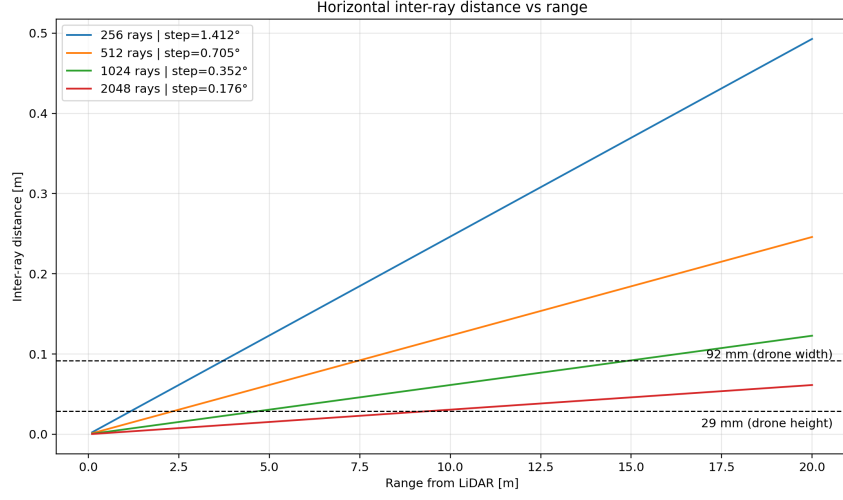
(a) LiDAR rate = 5Hz



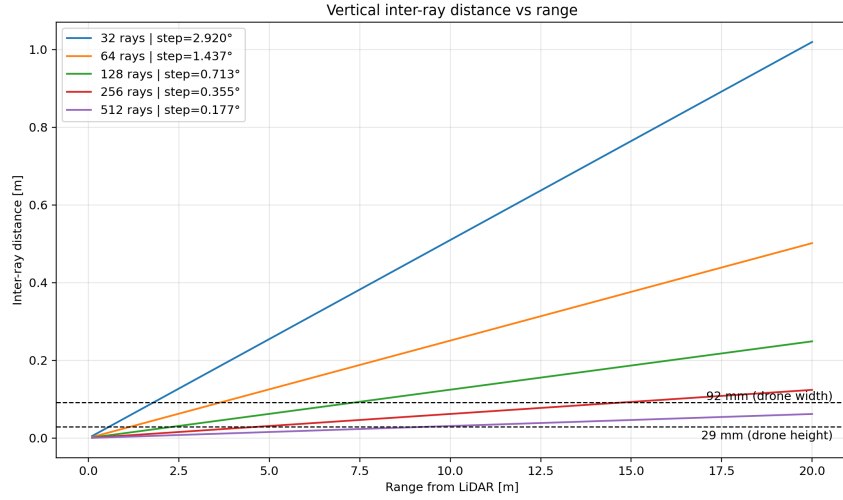
(b) LiDAR rate = 20Hz

Figure 5.10: Heatmaps of lost ratio for different pairs of LiDAR rays.

Therefore, when augmenting the flight distance of the drone the number of rays also needs to be augmented. The distance between rays at certain ranges can be computed based on the number of rays and the LiDAR FOV, and compared to the drone size, to determine a good fit for specific wanted working distances. This evolution of the inter-ray distance is plotted in **Figure 5.11**.



(a) Horizontal



(b) Vertical

Figure 5.11: Graphs of inter-ray spacing, for a LiDAR with $[-0.79, +0.79]$ vertical FOV, and $[-3.14, +3.14]$ horizontal FOV. The Crazyflie drone's dimensions are also shown as reference.

5.2.3 Performance for different drone speeds

The 2 different test speeds mentioned in **Section 5.2.1** are compared in this section. The algorithm was tested with different drone controller proportional coefficients (kp), in order to study the performance variations depending on the drone's speed. 2 variants were tested, as presented in **Table 5.2**. The drone was flying at a mean distance of $1.94m$ to the LiDAR (ranging from $0.70m$ to $2.67m$).

$kp [s^{-1}]$	Drone speed $[m.s^{-1}]$
1.5	0.18
2.0	0.28

Table 5.2: Approximate mean drone speed for different kp coefficient in the drone controller.

Figure 5.12 shows that the algorithm performs slightly better when the drone has a higher kp , this might be because the drone movement's are smoother with this kp and therefore it makes the prediction step more accurate. This is linked to the tuning of the drone controller. Also, bigger speed values make noise errors on them less significative, improving the estimation's accuracy.

Figures 5.13 and **5.14** show that a higher rate improves the estimation, but for this speed's magnitude both update rates could yield satisfying results.

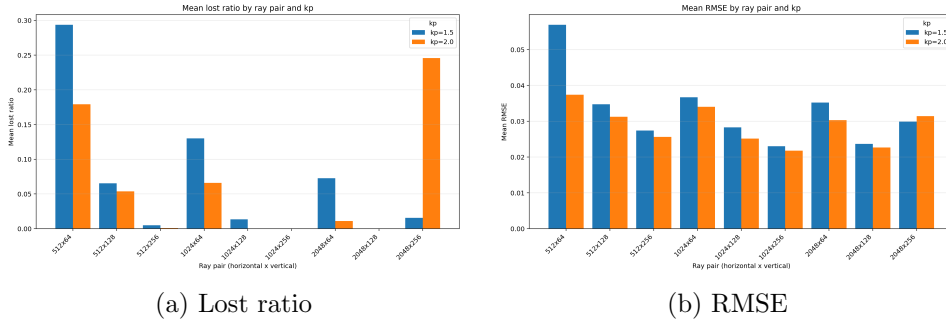


Figure 5.12: Bar graphs of lost ratio and RMSE for different pairs of LiDAR rays and kp values.

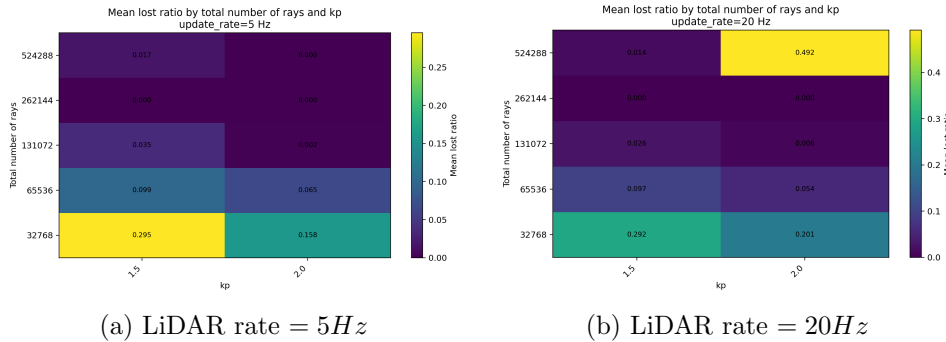


Figure 5.13: Heatmaps of lost ratio for different total numbers of LiDAR rays and different kp .

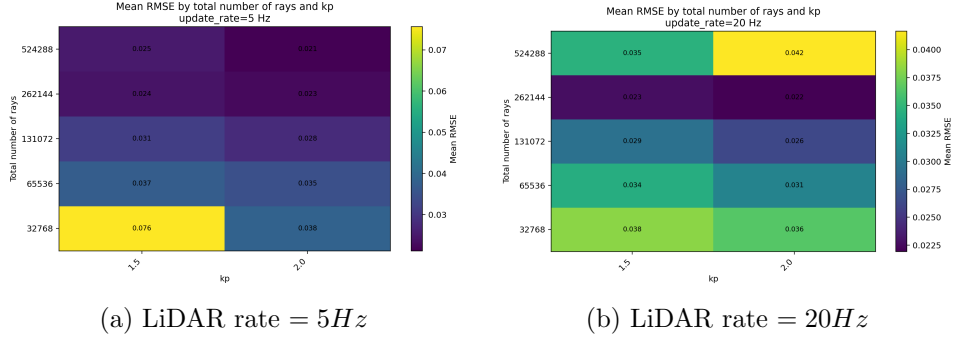


Figure 5.14: Heatmaps of RMSE for different total numbers of LiDAR rays and different kp .

5.3 Results for multi-drone tracking

Figure 5.15 shows an example of results obtained with 3 drones flying at the same time. The LiDAR used for this test had (128×512) rays, and an update rate of $5Hz$. The obtained generic metrics are shown in **Table 5.3**. In this example, one track was estimated for each true track for the whole duration of the experiment, the drones were never lost.

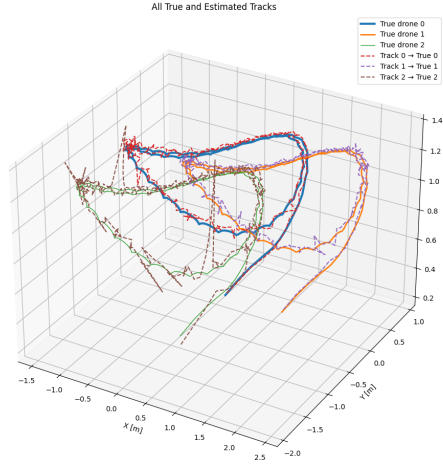
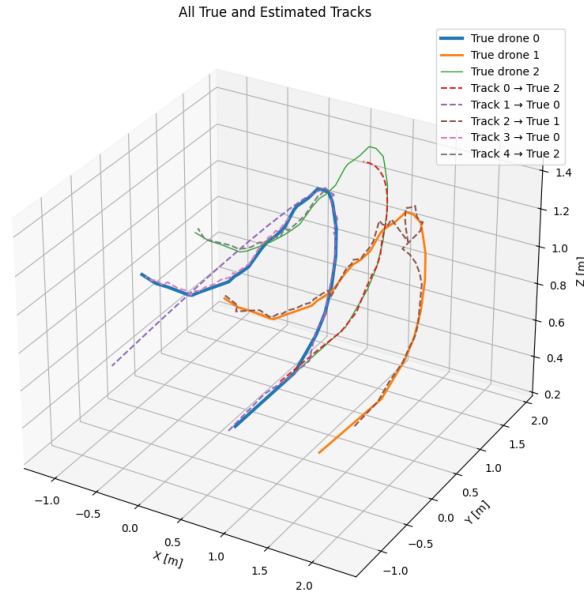


Figure 5.15: True and estimated trajectories of 3 simultaneously flying drones.

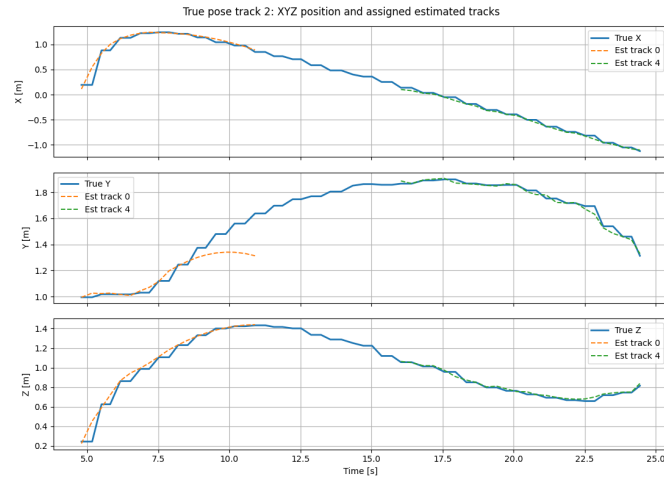
Table 5.3: Metrics by track for 3 drones, LiDAR : (128×512) , $5Hz$

Track ID	Mean RMSE [m]	Mean error [m]	Lost ratio
0	0.041	0.032	0
1	0.036	0.031	0
2	0.099	0.058	0

Figure 5.16a shows an example where some of the drones were lost and found back later on. The drones 0 and 2 each had 2 associated estimated tracks at different times in the simulation. This demonstrates the algorithm capability to recover from track loss and re-find drones after periods where they were lost. In **Figure 5.16b** it can clearly be seen when the algorithm lost track of drone 2 and got it back later on.



(a) 3D trajectory plots.



(b) Real and estimated x , y , and z of drone 2 over time.

Figure 5.16: Simulation with 3 drones, and multiple estimated track per drone (at different times).

Chapter 6 Conclusion

This project demonstrated the feasibility of localizing and tracking Micro Aerial Vehicles (MAVs) using a fixed LiDAR. A complete simulation framework was developed in Gazebo Sim and ROS2, with a configurable LiDAR model, a drone controller, localization and tracking algorithms, performance evaluation metrics, and an automated grid search pipeline. This simulation environment and pipeline enabled the evaluation of multiple different LiDAR configurations and flight conditions.

The designed localization algorithm combines background subtraction, adaptive region-of-interest selection, DBSCAN clustering, polynomial trajectory prediction, and Kalman filtering. The results show that accurate localization can be achieved for a Crazyflie drone, with position errors on the order of a few centimeters when the LiDAR has a sufficient resolution. The grid search highlighted the strong impact of LiDAR specifications (mainly the number of rays) and drone-LiDAR distance on the tracking performance. The results therefore provide practical insights for selecting LiDAR characteristics according to the expected operating distance and desired localization performance.

The localization algorithm and simulation framework was also extended to support multiple simultaneously flying drones through a track association system. Preliminary results show the algorithm basic principle is functional, but more experiments would be required in more challenging situations, like drones crossing each other for example.

This approach tried to prepare for more realistic conditions by simulating noise and taking multiple background frames during initialization for instance, but as the project was conducted entirely in simulation there would still be a simulation to reality gap to bridge when implementing it in real life. The movements in a real background might impact the algorithm performance, whose robustness would need to be more extensively tested and improved.

Additionally, the difference between mechanical or solid state LiDARs was not directly studied as the simulation did not replicate the physical internal operating principles of the sensor. In practice, these sensor types exhibit different scanning patterns, temporal sampling characteristics, and measurement distributions, which may influence the quality of drone local-

ization.

Future work could therefore focus on transferring the system to real LiDAR platforms and validating the approach experimentally. Additional improvements could also be done to reduce the computational cost of the algorithm, allowing higher-resolution LiDARs and higher update rates to be processed in real time.

Potential improvements include parallelization, dynamic multi-frequency scan integration[7] and more advanced multi-object tracking techniques.

The grid search studied classic LiDAR specifications which are often found in commercial LiDARs such as the Livox Mid-360S, or the Ouster OS0. The Ouster OS0 has a 90° vertical FOV and complete horizontal sweep like the LiDAR implemented in the simulation. Its resolution is configurable (512 to 4096 horizontal rays, and 32 to 128 vertical rays), as well as its rotation rate (5 to 40Hz).

Overall, this project provides a complete framework for evaluating a designed drone localization algorithm using a LiDAR. It demonstrates that reliable tracking of small drones is achievable with a fixed ground-based LiDAR when an appropriate compromise is found between sensor resolution, update rate, operating distance, and available computational resources.

Appendix A Code

The code for this project can be found on GitHub :
https://github.com/amandine-mnr/SP2_LiDAR_Drone.git

Bibliography

- [1] Livox Technology, “Which LiDAR Scanning Pattern is Better? Repetitive or Non-repetitive.,” 2022.
- [2] S. Dogru and L. Marques, “Drone detection using sparse lidar measurements,” *IEEE Robotics and Automation Letters*, vol. 7, pp. 3062–3069, April 2022.
- [3] H. Sier, X. Yu, I. Catalano, J. P. Queralta, Z. Zou, and T. Westerlund, “Uav tracking with lidar as a camera sensor in gnss-denied environments,” 2023.
- [4] Z. Luo, G. Zhang, C. Zhou, Z. Wu, Q. Tao, L. Lu, and S. Lu, “Modeling continuous motion for 3d point cloud object tracking,” 2024.
- [5] R. R. Pinho, J. M. R. S. Tavares, and M. V. Correia, “Efficient approximation of the mahalanobis distance for tracking with the kalman filter,” *International Journal of Simulation Modelling*, 2007.
- [6] Bitcraze, “crazyflie-simulation,” 2026.
- [7] I. Catalano, S. Ha, X. Yu, T. Westerlund, and J. P. Queralta, “Uav tracking with solid-state lidars: Dynamic multi-frequency scan integration,” pp. 417–424, 2023.